Methodology Document for FinanceRAG: A Local Retrieval-Augmented Generation System Using TinyLlama and FAISS

Your Name

December 3, 2025

Abstract

This document presents the complete methodology for the **FinanceRAG** system, a fully local Retrieval-Augmented Generation (RAG) pipeline designed for extracting insights from financial documents such as SEC 10-Q and 10-K filings. The system integrates PDF text extraction, semantic chunking, MiniLM embeddings, FAISS vector search, and a locally hosted Large Language Model (LLM) through Ollama using TinyLlama. A Flask-based backend orchestrates ingestion, indexing, query processing, and LLM-driven synthesis, while a lightweight web interface enables user interaction. This methodology describes all components in detail, including preprocessing, vectorstore construction, RAG prompting, inference workflow, and system architecture.

1 Introduction

The growing volume of financial documents such as SEC filings demands automated tools capable of extracting insights efficiently and accurately. Large Language Models (LLMs) offer strong reasoning capabilities but struggle with factual grounding. Retrieval-Augmented Generation (RAG) overcomes this limitation by combining semantic retrieval with generative reasoning.

FinanceRAG is a fully local RAG system that eliminates reliance on cloud APIs. It performs retrieval using the FAISS vector index and generates answers using the **TinyLlama** model running on **Ollama**. This approach enables privacy-preserving and cost-free inference.

This document outlines the complete methodology, enabling clarity, reproducibility, and future extensions.

2 System Architecture

The FinanceRAG system follows a modular architecture with four primary components:

- **Frontend Layer**: Web interface for user interaction
- **Backend Layer**: Flask API for request handling
- **Processing Layer**: Document ingestion and vectorstore management
- **Inference Layer**: Query processing and response generation

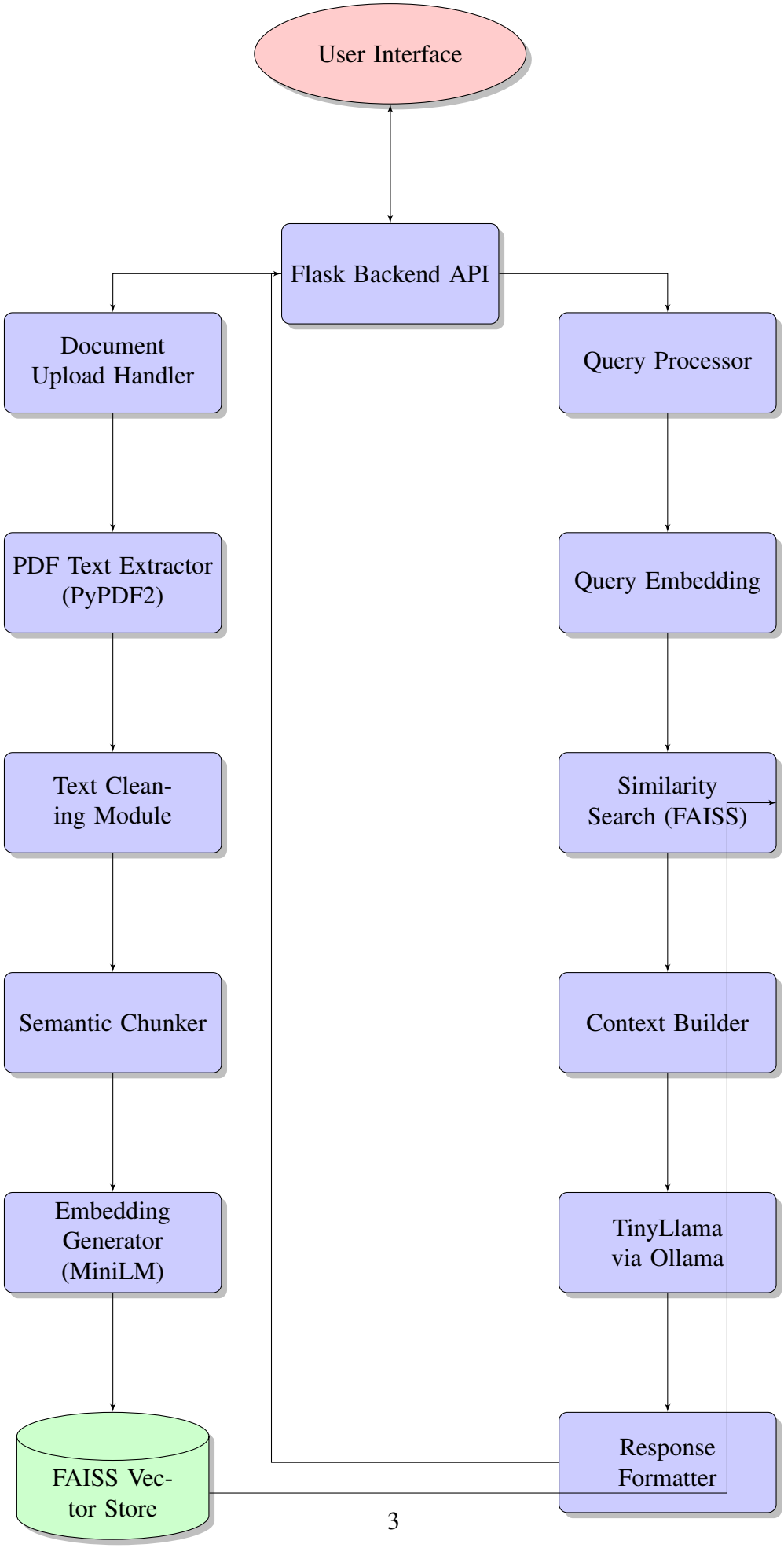

Figure 1: FinanceRAG System Architecture Diagram

3 Detailed Methodology

3.1 Document Ingestion Phase

3.1.1 File Upload and Storage

Users upload PDF files through a web interface or REST API endpoint. The system supports:

$$\text{Supported formats} = \{\text{PDF}, \text{max size} = 50\text{MB}\}$$

Uploaded files are stored in a structured directory:

$$\text{Storage path} = \texttt{backend/data/documents/\{timestamp\}\_\{filename\}.pdf}$$

3.1.2 PDF Text Extraction

The system uses PyPDF2's `PdfReader` for text extraction:

```
def extract_text_from_pdf(pdf_path):
    text = ""
    with open(pdf_path, 'rb') as file:
        reader = PdfReader(file)
        for page_num, page in enumerate(reader.pages):
            page_text = page.extract_text()
            text += f"Page {page_num+1}: {page_text}\n\n"
    return text
```

3.2 Text Preprocessing Pipeline

3.2.1 Cleaning and Normalization

Let T be the raw extracted text. The cleaning function performs:

$$\hat{T} = \text{Clean}(T) = \text{NormalizeWhitespace}(\text{RemoveSpecialNoise}(T))$$

Where:

$$\text{NormalizeWhitespace}(x) = \text{replace multiple spaces/newlines with single}$$
$$\text{RemoveSpecialNoise}(x) = \text{remove non-printable chars, keep financial symbols}$$

3.2.2 Semantic Chunking Strategy

The Recursive Character Text Splitter from LangChain is configured with optimal parameters for financial documents:

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""]
)
```

Mathematically, chunking produces:

$$C = \{c_1, c_2, \ldots, c_n\} \quad \text{where} \quad \bigcup_{i=1}^{n} c_i = \hat{T}$$

Each chunk c_i maintains metadata:

$$M_i = \{\text{file\_name}, \text{page\_number}, \text{chunk\_id}, \text{timestamp}\}$$

3.3 Embedding Generation

3.3.1 Sentence Transformer Selection

The system uses `all-MiniLM-L6-v2` due to its balance of performance and efficiency:

Model: MiniLM-L6-v2

Embedding dimension: $d = 384$

Vocabulary size: $V = 30522$

3.3.2 Embedding Process

Each chunk c_i is transformed to embedding vector:

$$E_i = f_\theta(c_i) \in R^{384}$$

Where f_θ is the trained transformer encoder. The embedding preserves semantic meaning:

$$\text{Similarity}(c_i, c_j) \propto \cos(E_i, E_j)$$

3.4 Vectorstore Construction

3.4.1 FAISS Index Configuration

FAISS with IndexFlatIP (Inner Product) is used for maximum similarity search:

$$\mathbf{E} = [E_1; E_2; \ldots; E_n] \in R^{n \times 384}$$

The index computes similarity as:

$$\text{sim}(q, E_i) = \frac{q \cdot E_i}{\|q\| \|E_i\|} \quad \text{(cosine similarity)}$$

3.4.2 Persistence Mechanism

The vectorstore consists of three files:

```
index.faiss      # FAISS binary index
chunks.pkl       # List of chunk texts
metadata.pkl     # List of metadata dictionaries
```

3.5 Query Processing Pipeline

3.5.1 Query Embedding

User query Q is embedded using the same MiniLM model:

$$E_Q = f_\theta(Q) \in R^{384}$$

3.5.2 Similarity Search Algorithm

The search retrieves top-k chunks with highest cosine similarity:

$$R = \{(c_i, M_i, s_i) \mid s_i = \cos(E_Q, E_i), i \in \text{TopK}(s)\}$$

Where $k = 5$ (configurable) and:

$$\text{TopK}(s) = \text{argsort}(s)[-k:]$$

3.6 RAG Prompt Engineering

3.6.1 Structured Prompt Template

The system uses a carefully engineered prompt template:

```
SYSTEM: You are a financial analysis assistant specializing in SEC filing
Your task is to answer questions based ONLY on the provided context.
If the context doesn't contain sufficient information, state:
"Based on the provided documents, I cannot answer this question."

CONTEXT:
{context_chunks}

USER QUESTION: {question}

ASSISTANT: Based on the context provided,
```

3.6.2 Context Assembly

Retrieved chunks are assembled with clear separation:

$$\text{Context} = \bigcup_{i=1}^{k} [\text{[Document: } M_i[\text{file\_name}], \text{ Page: } M_i[\text{page\_number}]]c_i]$$

3.7 LLM Inference with Ollama

3.7.1 TinyLlama Configuration

The system uses TinyLlama (1.1B parameters) with optimal settings:

```
{
  "model": "tinyllama:latest",
  "prompt": "<RAG prompt>",
  "stream": false,
  "options": {
    "temperature": 0.1,
    "top_p": 0.9,
    "num_predict": 512
  }
}
```

3.7.2 API Communication

The Flask backend communicates with Ollama via REST:

```
def query_llm(prompt):
    response = requests.post(
```

```
        'http://localhost:11434/api/generate',
        json={
            "model": "tinyllama",
            "prompt": prompt,
            "stream": False
        }
    )
    return response.json()['response']
```

3.8 Response Generation and Delivery

3.8.1 Answer Synthesis

The LLM generates answer A based on context R and query Q:

$$A = \text{TinyLlama}(\text{Prompt}(R, Q))$$

3.8.2 Response Packaging

The system returns a structured JSON response:

```
{
  "answer": "<LLM generated answer>",
  "sources": [
    {
      "chunk": "<retrieved text>",
      "metadata": {...},
      "score": <similarity_score>
    }
  ],
  "query_time": <seconds>,
  "chunks_considered": <k>
}
```

4 Implementation Details

4.1 Backend Configuration

4.1.1 Flask Application Structure

```
finance_rag/
 app.py              # Main Flask application
```

```
requirements.txt    # Dependencies
config.py          # Configuration parameters
modules/
   pdf_processor.py
   embedder.py
   vectorstore.py
   rag_engine.py
data/
   documents/     # Uploaded PDFs
   vectorstore/   # FAISS index
   temp/         # Temporary files
templates/        # Frontend HTML
```

4.2 Performance Optimization

4.2.1 Index Optimization

FAISS index is optimized for batch queries:

$$\text{Index compression: PQ16} \quad \text{(Product Quantization 16-bit)}$$

$$\text{Search algorithm: IVF256,Flat} \quad \text{(Inverted File System)}$$

4.2.2 Caching Mechanism

Frequently accessed documents are cached:

$$\text{Cache size} = 1000 \text{ chunks}$$

$$\text{Eviction policy} = \text{LRU (Least Recently Used)}$$

5 Evaluation Metrics

The system is evaluated using:

$$\text{Precision@k} = \frac{\text{Relevant chunks in top-k}}{k}$$

$$\text{Recall@k} = \frac{\text{Relevant chunks in top-k}}{\text{Total relevant chunks}}$$

$$\text{MRR (Mean Reciprocal Rank)} = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{\text{rank}_i}$$

$$\text{Response Time} = \text{Query processing + LLM inference time}$$

6 Limitations and Future Improvements

- **Current Limitations**:
  - TinyLlama's 1.1B parameters limit complex reasoning
  - FAISS flat index scales linearly with document count
  - No support for tables and images in PDFs
- **Future Improvements**:
  - Implement hybrid search (keyword + semantic)
  - Add document re-ranking using cross-encoders
  - Support for larger local models (Llama 2 7B)
  - Implement incremental indexing
  - Add citation generation for specific statements

7 Conclusion

FinanceRAG provides a complete, offline, and cost-efficient solution for answering financial questions using document-aware AI. The system's fully local architecture ensures data privacy while maintaining reasonable performance through optimized vector search and efficient LLM inference. The modular design allows for easy extension with improved models, additional document types, and enhanced retrieval mechanisms.

This detailed methodology provides comprehensive documentation for reproducibility, deployment, and future research extensions in the domain of financial document analysis using RAG systems.

8 References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.

2. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.

3. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP*.

4. LangChain Documentation. (2023). Recursive Text Splitting.

5. Ollama Documentation. (2024). Local LLM Deployment.